

# Making Watermark Survive Model Extraction Attacks in Graph Neural Networks

Haiming Wang<sup>1</sup> Zhikun Zhang<sup>2</sup> Min Chen<sup>2</sup> Shibo He<sup>1,3</sup>

<sup>1</sup>Zhejiang University

<sup>2</sup>CISPA Helmholtz Center for Information Security

<sup>3</sup>Key Laboratory of Collaborative Sensing and Autonomous Unmanned Systems of Zhejiang Province

**Abstract**—Collecting graph data is costly and well-trained graph neural networks (GNNs) are viewed as intellectual property. To make better use of GNNs, they are used to provide cloud-based services. However, models on cloud-based services may be leaked under model extraction attacks. Adversaries can extract an imitation model by simply querying the GNNs on the cloud-based services. To protect GNNs, watermarks are embedded in the models. However, the watermarks can be removed by the model extraction attacks. To address this issue, we propose adding a watermark that cannot be ignored by queries from the model extraction attacks. Concretely, we add the soft nearest neighbor loss to the loss function of the watermark embedding process to merge the distributions for the normal tasks and watermarks. We also observe that the watermark brings a performance loss to GNNs and propose an optimization method to maintain the model performance. We evaluate our method on multiple real-world datasets to demonstrate the superiority of the method.

**Index Terms**—graph neural network, watermark

## I. INTRODUCTION

Graph ubiquitously exists in the real world. As the state-of-the-art approach to handle graph data, the graph neural network (GNN) is widely used in areas such as wireless networks [6]. Collecting graph data to train a graph neural network is costly. For example, graph neural networks trained on molecular graphs are useful in drug discovery; however, analyzing molecular graphs costs a lot. Therefore, a well-trained model is intellectual property. To make better use of the model, the model owner could allow other users to use the model through cloud services, which is called *machine learning as a service* (MLaaS). MLaaS provides cloud devices and models in some platforms (such as AWS Machine Learning) by running models for users. More specifically, users transfer data to the platform as input for the graph neural network. The model calculates results on the platform's devices, and finally, the output is given back to the users.

Every coin has two sides. On the one hand, MLaaS give users who do not hold data the chance to use well-trained graph neural networks. On the other hand, MLaaS introduces an attack surface for the attacker. In the *model extraction attacks* [8], the attacker only needs the ability to query the model. The attacker can “copy” a model by catching the relationship between some inputs and outputs of the model. The most straightforward approach to defend against model extraction is

forbidding the attacker to touch the output of the victim model by setting a firewall. However, distinguishing all the attackers from normal users is hard. Meanwhile, a single successful attack can make any firewall into a Maginot line, since the copies of the leaked model will spread widely through the Internet.

A more practical method is adopting the *watermark* paradigm to show the ownership of a model. To claim ownership of a model, the model owner must show that she has knowledge of the model that is only known by the owner herself. For graph neural networks, the knowledge is usually some behavior only held by the watermarked model. The owner queries the model on some specific inputs and the surprising prediction returned by the model meets the knowledge of the owner. Models can acquire watermarks by learning such behavior through training. Existing works [4], [10] propose several algorithms to embed watermarks in models. These works focus on improving the performance of watermarks on the watermark-embedded models.

However, the simply embedded watermark can be removed by model extraction. Watermarking techniques take advantage of the deep learning models' ability to learn different distributions at the same time: One is the distribution for normal tasks; Another is the distribution to remember the behavior showing the watermark. The models extracted from the watermarked model do not naturally learn the distribution for the watermark. In the extraction attack, the attacker queries the model in the distribution for normal tasks. Since the distributions for normal tasks and the watermark are usually separated, the extracted model only learns the distribution for normal tasks while the distribution for the watermark is ignored. Then the extracted model will not show the watermark. How to embed a watermark that is unable to remove by the model extraction attacks without affecting the performance of the watermarked model too much is the main challenge and motivation of our method.

**Our Contributions.** In this paper, we propose a technique that adds a model-extraction-resistant watermark to graph neural networks on cloud services. Concretely, we change the loss function of the watermark embedding process to prevent the distribution of the watermark from being ignored by merging

different distributions learned. The watermark embedding is training a model using specific inputs and outputs to make the model have the behavior (having specific outputs given specific inputs) showing the watermark. First, to merge the distributions of the normal tasks and the watermark, we apply the soft nearest neighbor loss [3] to the training process. Then we find that the watermark may harm the model's performance. Therefore, we propose a method to optimize the watermark embedding so it does less harm to the model performance.

We evaluate the approach on two datasets MSRC-9 and ENZYMES. We demonstrate that our approach can add a watermark resistant to the model extraction attacks to the graph neural networks on cloud services. We further showed that our optimization method can improve the model performance through evaluations.

To summarize, the main contributions of this paper are three-fold:

- We propose a method to add a model-extraction-resistant watermark to the graph neural networks on cloud services.
- We propose an optimization-based technique to improve the performance of the watermarked graph neural networks.
- We perform evaluations to show the effectiveness of our proposed methods.

## II. PRELIMINARIES

### A. Graph Neural Network

Graph structures commonly exist in many real-world data such as social networks, wireless networks, and financial networks. Traditional machine learning models do not perform well on these data, as they are less effective in making use of network topology. Recently, graph neural networks (GNNs) has emerged as a family of representation learning models that encode the network topology. GNNs adopt the deep learning model framework and learn network topology by aggregating information from nodes' neighbors using methods like network propagation.

GNN can be used for node-level tasks, edge-level tasks, and graph-level tasks. For node-level and edge-level tasks, we can predict some properties of nodes in a graph and edges in a graph, respectively. For the graph-level tasks, we can predict some properties of a whole graph, instead of some inner elements of a graph. In this paper, we focus on graph-level tasks, and the graph data we use are undirected. According to the ways of graph propagation, there are many GNN models, such as GCN, GAT, and GraphSage. In this paper, we use GraphSage as our target model.

### B. MLaaS and Model Extraction Attack

In MLaaS, model owners put their models on the cloud and open APIs for customers of their service. A customer can submit their graph data to the model on the cloud and obtain classification results from the model. The classification process of the model happens on the cloud.

Model extraction attack is becoming an appealing attack method with the popularity of deep learning models. In the

model extraction attack, the model that the attacker wants to steal is called *victim model*. The goal of the model extraction attack is to recover a similar or even the same model as the victim model. For an attacker whose target is MLaaS, the only ability they need is to query the model on the cloud. The rationale behind the model extraction attack is to catch the process that converts input data to output. A typical attack is as follows. To launch the model extraction attack, the attacker has dataset  $\mathcal{D}'$  analogous to the training dataset  $\mathcal{D}$  of the victim model and uses them to query the victim model. The victim model returns the predicted labels. The attacker then trains a model using  $\mathcal{D}'$  and the returned labels to mimic the behavior of the victim model. The model trained by the attacker is called *extracted model*. We denote the victim model and extracted model as  $M$  and  $M_e$ , respectively. For a dataset  $G$ , the objective of the model extraction attack is to minimize  $\sum_{x \in G} |M(x) - M_e(x)|$ . The extracted model has very similar behavior to the victim model if the extraction attack is successful.

## III. METHODOLOGY

### A. Method Overview

Considering that preventing a stealing action is difficult, we choose to use the watermark to show the model **ownership** with which the model owner can rely on the law to protect her intellectual property. The rationale behind the watermark technique is to hide information about ownership as metadata within the digital property. The hidden information is only held by the watermarked object and is only known by the owner. When the owner wants to verify the ownership of an object, she tries to reveal the information from the object, and show that the information agrees with the special knowledge she holds. For graph neural networks, the information is the watermarked models' special behaviors that are different from the non-watermarked models. More specifically, the special behaviors are the models' certain outputs given certain inputs. The certain inputs and outputs that the model owner wants the watermarked model to hold are called the key inputs and the key labels of the watermark. The set of key inputs and the key labels are denoted as  $\{x_i^k\}$  and  $\{l_i^k\}$ . The  $i$ th key label  $l_i^k$  corresponds to the  $i$ th key input  $x_i^k$ .

The watermark framework includes three phases:

- **Key Inputs and Labels Generation.** The owner first generates several key input-label pairs  $(x_i^k, l_i^k)$ . The pairs  $(x_i^k, l_i^k)$  are outliers only known by the owner. If  $(x_i^k, l_i^k)$  is not an outlier, the non-watermarked model will output  $l_i^k$  given  $x_i^k$ ; therefore, the model's behavior is not special knowledge and can be known by anyone who has a non-watermarked model. In this case, the owner fails to verify her ownership.
- **Watermark Embedding.** The watermark embedding process is training the model with  $x_i^k$  and  $l_i^k$ . The intuition here is to make the model learn the distribution of the key inputs and labels. If the distribution of the key inputs and labels is learned successfully,  $M$  will have output  $l_i^k$  given  $x_i^k$ .

- **Ownership Verification.** To show the ownership of the watermarked model  $M$ , the owner submits the key inputs  $x_i^k$  to  $M$  one by one and compares the outputs with the key labels. If the model owner suspects that another model is a stolen copy of her model, she can test the suspect model with the key inputs  $x_i^k$  to check if there is any watermark in this model. For one key input  $x_i^k$ , if the model's outputs agree with the corresponding key label  $l_i^k$ , we say that the model remembers the key input-label pair  $(x_i^k, l_i^k)$ . If the model remembers most input-label pairs, the watermark is considered to exist on the suspect model and the model owner shows her ownership of the suspect model.

To defend against the model extraction attack, we want to design a watermark-adding method that can retain in the extracted model. We face three challenges: How to design  $x_i^k$  and  $l_i^k$ ? How to keep the watermark in the extracted model? How to reduce the performance loss of  $M$  caused by the watermark?

### B. Strawman Watermark

First, we create key inputs and their key labels. To make the model have output  $l_i^k$  given key input  $x_i^k$ , the model needs to be trained using  $x_i^k$  with label  $l_i^k$ . Suppose that we need  $N^k$  key input-output pairs  $(x_i^k, l_i^k)$  to add the watermark, we discuss how to design every  $x_i^k$  and its corresponding  $l_i^k$ .

**Generating the Key Inputs.** We use the random graph models to create key inputs which are outliers of the training dataset. Part of the key input comes from the training dataset, and the other parts come from outlier data. We first initialize  $x_i^k$  as an empty graph with  $N$  nodes. Then we randomly choose  $N^t$  node from  $x_i^k$ . These chosen nodes form a subgraph  $G^t$ . Then topology and features of  $G^t$  and the rest part of  $x_i^k$  are created by different methods.

- **Random Part  $G^t$ .** The topology of  $G^t$  is generated using a random graph model such as the ER random graph model while features of  $G^t$  are generated randomly.
- **Rest Part of  $x_i^k$ .** We randomly choose a graph  $G'$  that has the same number of or more nodes than  $x_i^k$  from the training dataset. After that, we randomly choose a subgraph  $G^o$  with  $N$  nodes from  $G'$ . The topology and node features of  $x_i^k$  except for  $G^t$  adopt the topology and features of  $G^o$ .

The detailed topology generation algorithm is illustrated in Algorithm 1. The time complexity of generating all the key inputs is  $\mathcal{O}(N^k |N_{ave}|)$ , where  $N_{ave}$  is the average number of the node of graphs in  $\mathcal{D}$ .

**Generating the Key Labels.** The key labels are generated following one rule: the label  $l_i^k$  of  $x_i^k$  should be different from the label predicted by the non-watermarked model. If  $l_i^k$  is chosen as the prediction of the non-watermark model of  $x_i^k$ , the model owner cannot verify her ownership since the output is not specially linked to the owner. We choose the label in a straightforward way: We train a non-watermarked model using the training dataset and use the trained model to predict a label

---

### Algorithm 1: Key Input Topology Generation

---

**Input:** The training dataset  $\mathcal{D}$ , key input  $x_i^k$  as an empty graph, the number of nodes to be randomized  $N^t$ ;

**Output:** Key input  $x_i^k$  with randomized topology;

- 1 Randomly sample a subgraph  $G^o$  with  $N$  nodes from a randomly chosen graph in  $\mathcal{D}$ ;
  - 2 Randomly sample a subgroup  $G^t$  with  $N^t$  nodes from  $x_i^k$ ;
  - 3 Generate a graph  $G^r$  with random topology and  $N^t$  nodes;
  - 4 Create a bijection  $f_1$  that maps nodes in  $x_i^k$  to nodes in  $G^o$ ;
  - 5 Create a bijection  $f_2$  that maps nodes in  $G^t$  to nodes in  $G^r$ ;
  - 6 **for** node  $v_1 \in V(x_i^k)$  **do**
  - 7     **for** node  $v_2 \in V(x_i^k) - v_1$  **do**
  - 8         **if** Both  $v_1$  and  $v_2 \in G^t$  **then**
  - 9             **if** There is an edge between  $f_2(v_1)$  and  $f_2(v_2)$  in  $G^r$  **then**
  - 10                 Add an edge between  $v_1$  and  $v_2$ ;
  - 11         **else**
  - 12             **if** There is an edge between  $f_1(v_1)$  and  $f_1(v_2)$  in  $G^o$  **then**
  - 13                 Add an edge between  $v_1$  and  $v_2$ ;
- 

$l'$  for  $x_i^k$ .  $l_i^k$  is randomly chosen among all possible labels except for  $l'$ .

The key inputs and labels are used to train the graph neural networks with the training data. After training, the victim model  $M$  has a watermark since it has output  $l_i^k$  given input  $x_i^k$ . We call it a strawman watermark, and a model has a strawman watermark if it is simply trained using generated key inputs and labels.

### C. Model-extraction-resistant Watermark

The model extraction attack can extract a “copy” model  $M_e$  of the victim model  $M$ . We hope the watermark can retain on  $M_e$  so that the owner of model  $M$  can verify ownership on the model extracted from  $M$ . Unfortunately, the strawman watermark can be removed by the model extraction attack (see the evaluation results in Section IV-B).  $M$  usually learns the normal data and the watermark as two distributions. If  $M$  has sufficient capacity, it learns both two distributions independently. The independence of the two distributions makes  $M_e$  “forget” the watermark. In the model extraction attack, the attacker only touches on the distribution of the training data. More specifically, the attacker only queries  $M$  using data from the same distribution of the training dataset  $\mathcal{D}$ . Only the model parameters related to the distribution of the training data are activated in  $M$ . Thus, the label returned by  $M$  only fits the distribution of the training dataset  $\mathcal{D}$  while the distribution of the watermark is ignored. Therefore, a watermark embedded

by the normal training process will be removed from the extracted model.

To make the watermark resistant to model extraction attacks, we use the key inputs  $x_i^k$  and labels  $l_i^k$  in the strawman watermark, but change the embedding process to merge the distributions for the normal tasks and the watermark. More specifically, we add the soft nearest neighbor loss [3] in the watermark embedding process. The soft nearest neighbor loss (SNNL) is used to measure how close the distributions learned by the  $M$  for different groups of data are.

$$SNNL(X, Y, T) = -\frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \log \left( \frac{\sum_{\substack{j=1 \\ j \neq i}}^{|\mathcal{D}|} e^{-\frac{\|x_i - x_j\|^2}{T}}}{\sum_{\substack{k=1 \\ k \neq i}}^{|\mathcal{D}|} e^{-\frac{\|x_i - x_k\|^2}{T}}} \right) \quad (1)$$

SNNL is calculated as Equation 1.  $X$  is the dataset and  $Y$  is the set of groups of data (such as training data and key input-output). The SNNL compares the distances between data points of the whole dataset with the distances between data points of the same group. When the SNNL value is large, on average, two data points in the whole dataset are relatively closer to two data points in the same group. In this case, the distributions of the different groups of data are close.  $T$  is the parameter to decide whether to put a higher weight on the larger distances or smaller distances.

Distributions are merged if they are close. In the training process of  $M$ , we add the SNNL to the loss function in the training process of the watermark embedding to make the distributions close.

$$Loss_{MER} = Loss_M - SNNL([\mathcal{D}, (x_i^k, l_i^k)], Y, T) \quad (2)$$

The set  $Y$  contains two different groups: training data (data in  $\mathcal{D}$ ) and key input-label pairs. The model obtains a model-extraction-resistant watermark after watermark embedding with SNNL.

#### D. Optimized Model-extraction-resistant Watermark

After training with SNNL, the watermark is resistant to model extraction attacks. However, training to embed the watermark degrades the performance of  $M$  since the training data distribution remembered by  $M$  is changed by the key input-label pairs.

To alleviate performance degradation, we optimize the random part of  $x_i^k$ . Considering that the topology of a graph is discrete and binary, using straightforward gradient descent is impossible. Instead, we set DNN models to optimize the random part of  $x_i^k$ . We use the first DNN model to optimize the topology and denote it by  $M_{topo}$ .  $M_{topo}$  consists of two  $N^t \times N^t$  fully-connected layers. The activation function of the first layer is ReLU. The input of  $M_{topo}$  is an adjacency matrix and the output is a matrix of the same scale. The values in the

#### Algorithm 2: Optimizing Key Inputs

---

**Input:** Training dataset  $\mathcal{D}$ , set of key input-label pairs  $S$ , number of optimization turns  $T_{opt}$  ;

**Output:** Set of optimized key input-label pairs  $S$ ;

- 1 Set the set of optimized key inputs  $S$  as an empty set ;
- 2 Initialize  $M_{topo}$  and  $M_{feat}$ ;
- 3 **for**  $j \in 1 \cdots T_{opt}$  **do**
- 4   **for**  $(x_i^k, l_i^k) \in S$  **do**
- 5      $m_{topo} \leftarrow$  the adjacency matrix of the random part of  $x_i^k$ ;
- 6      $m_{topo} \leftarrow M_{topo}(A_{topo})$ ;
- 7     Change the topology of the random part of  $x_i^k$  according to  $m_{topo}$ ;
- 8      $m_{feat} \leftarrow$  the features of the random part of  $x_i^k$ ;
- 9      $m_{feat} \leftarrow M_{topo}(m_{feat})$ ;
- 10    Change the features of the random part of  $x_i^k$  according to  $m_{feat}$ ;
- 11    Train a watermarked model  $M_{opt}$  using  $\mathcal{D}$  and  $S$ ;
- 12    Calculate loss function  $Loss_{opt}(M_{opt}, \mathcal{D})$  according to Equation 3;
- 13    Update parameters in  $M_{topo}$  and  $M_{feat}$  by backward propagation of  $Loss_{opt}$ ;

---

output matrix are assigned to 0 or 1 by a threshold function to make it an adjacency matrix. Similarly, we set another DNN model  $M_{feat}$  to optimize the features of the randomized part of  $x_i^k$ .  $M_{feat}$  consists of two  $K \times K$  fully connected layers, where  $K$  is the dimension of the features of  $x_i^k$ . The activation function is also ReLU. We set the same loss function for both  $M_{topo}$  and  $M_{feat}$ . That is the cross entropy of predicted labels and real labels on the training dataset.

$$Loss_{opt}(M, \mathcal{D}) = CE(M(\mathcal{D}), labels) \quad (3)$$

However, the optimization is still very hard since directly calculating the loss function and using backward propagation to update the parameters of  $M_{topo}$  together with  $M_{feat}$  are almost not possible. This is because the forward propagation from  $M_{topo}$  or  $M_{feat}$  to the loss function is too complex. Forward propagation begins with calculating the optimized  $x_i^k$ , then goes to the training of the watermarked model, and finally ends with the calculation of the loss function based on the labels predicted by the trained watermarked model. The forward propagation of  $M_{topo}$  and  $M_{feat}$  contains the training process of the watermarked model. Therefore, if we want to conduct backward propagation, the gradient will be too complex to calculate. Instead, we choose to decouple the watermark model training and forward propagation in a bi-level way. More specifically, the optimization is divided into many bi-level turns. In every turn, we do watermark model training and backward propagation separately. The details of the optimization are given in Algorithm 2.

After the optimization, we use the optimized key input-label pairs to embed a watermark with SNNL loss. We say this



model has an optimized model-extraction-resistant watermark.

#### IV. EVALUATION

##### A. Experimental Setup

**Datasets.** We run experiments on two widely used datasets: MSRC-9 and ENZYMES. They are all from the TUDataset [5]. Both datasets are randomly shuffled and divided into three non-overlapping datasets: the training dataset  $\mathcal{D}$ , the extraction attack dataset  $\mathcal{D}'$ , and the testing dataset  $\mathcal{D}_{test}$  by ratios of 0.5, 0.3, and 0.2, respectively.

**Models.** In our evaluation, we use one of the most popular state-of-the-art GNN model GraphSage [2] as the victim models  $M$ . In every watermark embedding process, the model is trained with the key inputs-label pairs for 200 epochs.

**Metrics.** Our metric in the evaluations is the average effectiveness of the watermark. The definition and calculation of the average effectiveness are introduced as follows. First, we define the watermark retaining rate of a single extracted model. Suppose  $M_e$  is extracted from  $M$ , we say the input-label pair  $(x_i^k, l_i^k)$  is retained if for input  $x_i^k$  the extracted model's predicted label  $M_e(x_i^k)$  is exactly  $l_i^k$ . The watermark retaining rate of a watermark can be defined as the proportion of the retained input-label pairs among all input-label pairs:

$$E_{sin}(M_e) = \frac{1}{|S|} \sum_{(x_i^k, l_i^k) \in S} \mathbb{1}_{(M_e(x_i^k) = l_i^k)} \quad (4)$$

$S$  is the set of all input-label pairs. We need to further confirm that the watermark is effective. We set two thresholds for the watermark. One is the watermark retaining rate on the non-watermarked model  $E_{sin}(M)$ . Another is the watermark retaining rate on the random guess model  $E_{sin}(M_r)$ , where  $M_r$  is a model that predicts a random label for any input. If  $E_{sin}(M_e)$  is both larger than  $E_{sin}(M)$  and  $E_{sin}(M_r)$ , we say this watermark is effective on  $M$ . In our evaluations, each experiment repeats 100 times, and the proportion of success watermark among all experiments is defined as the average effectiveness of the watermark  $E_{ave}$ .

##### B. Watermark Effectiveness

In this part, we conduct empirical evaluations to show that our proposed watermarking method is effective.

**Experimental Setup.** In the experiments, we use the Erdos-Renyi random graph model to initialize the topology of random parts in the key inputs. In the Erdos-Renyi random graph model, every edge between any two nodes in the graph exists by a certain probability. This probability is 0.5 in all of our experiments. The node number of the random part of the key inputs is an important parameter of the key inputs. The more nodes in the random part of the key inputs, the more significant impact the key inputs have on the victim model. We vary this parameter in our experiments by choosing the number of the random part as 3, 4, 5, 6, and 7, respectively. Another important parameter is the number of key inputs. We choose the number of key inputs according to the number of graphs in the training dataset. That is, the number of key inputs is

the integer part of the training dataset size times a ratio  $a$ . If not specifically mentioned,  $a$  is 0.1 in experiments. The number of optimization turns  $T_{opt}$  is set as 20 for all key inputs optimizations (see Algorithm 2).

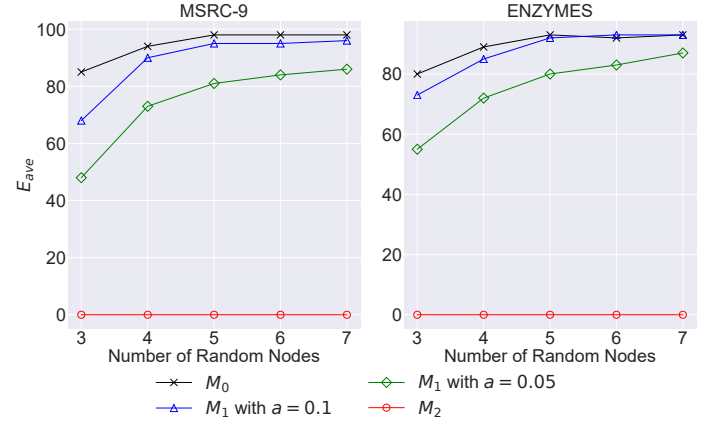


Fig. 1: Watermark effectiveness of different watermarked models. Columns stand for different datasets. In each subfigure, the y-axis stands for the average effectiveness of the watermark, and the x-axis stands for the number of nodes in the random part in the key inputs. A higher value indicates better performance.

**Results.** The watermark effectiveness is shown in Figure 1.  $M_0$  is the strawman-watermark embedded model.  $a$  is set as 0.1 in  $M_0$ .  $M_1$  is the optimized model-extraction-resistant model.  $M_2$  is the model extracted from  $M_0$ . We use  $M_1$  to conduct two groups of experiments:  $a = 0.1$  and  $a = 0.05$ . First, we compare the watermark effectiveness of  $M_0$  and  $M_2$ . The watermark effectiveness in  $M_0$  is greater than 0.8 for all results. This indicates that our proposed watermarking method can successfully add a watermark to the non-extracted model. However, in the extracted model  $M_2$ , the watermark effectiveness goes down to 0. The zero result of  $M_2$  shows that the strawman watermark (see Section III-B) can not retain on the extracted models, which agrees with our claim in Section III-C. To show the effectiveness of the soft nearest neighbor loss, we compare the results of  $M_1$  and  $M_2$ . We observe that  $M_1$  with  $a = 0.1$  has a watermark effectiveness of more than 0.9 in most cases, while  $M_2$  has zero watermark effectiveness. These results indicate that after training with SNNL, the distributions for normal tasks and the watermark merged, therefore the watermark retains on the extraction model.

Concerning the number of key input-label pairs, we compare the watermark effectiveness of  $M_1$  with  $a = 0.05$  and  $M_1$  with  $a = 0.1$ . On both two datasets,  $M_1$  with  $a = 0.1$  has better watermark effectiveness than  $M_1$  with  $a = 0.05$ , indicating that more key input-label pairs bring a stronger watermark. We think this is because more data make the model remember the distribution of data better. In general, the watermark effectiveness increases with the number of nodes in the random part. This happens because the key inputs

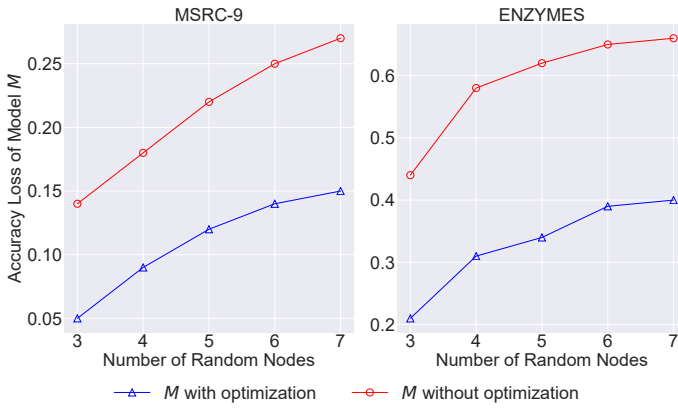


Fig. 2: Accuracy loss of watermarked models. Columns stand for different datasets. In each subfigure, the  $y$ -axis stands for the prediction accuracy loss of the watermarked model, and the  $x$ -axis stands for the number of nodes in the random part in the key inputs. A lower value indicates better performance.

are outliers further away from the training dataset when the number of nodes in the random part increases.

### C. Model Performance

In this section, we show that the key-graph optimized described in Section III-D can improve the model performance. **Experimental Setup.** We compare the loss of model performance with the optimized watermark and the non-optimized watermark. The loss in model performance of model  $M$  is defined as  $(|acc(M')| - |acc(M)|) / |acc(M')|$ , where  $M'$  is the non-watermarked model, and  $acc(\cdot)$  is the prediction accuracy of a model on the test dataset. The number of optimization turns  $T_{opt}$  is set as 20 for all key inputs optimizations.

**Results.** The results are shown in Figure 2. Comparing the performance loss of models watermarked with and without optimization, we can observe that the models watermarked by key inputs with optimization have better accuracy. In the other words, the optimization method we proposed in Section III-D can raise the model performance.

In general, the performance loss increases with the number of nodes in the random part, indicating that the model accuracy decreases more by the key inputs having more nodes in the random part. This is in line with our claims that that merging different distributions will affect the model accuracy since the distribution of the key inputs with more nodes in the random part is more different from the distribution of the training dataset.

## V. RELATED WORKS

Attacks and defenses on graph neural networks have been a hot topic in recent years. Zhang et al. [9] study inference attacks against graph neural networks. They infer important attributes such as the number of nodes, the number of edges, and graph density with very high accuracy from graph embeddings. They also propose a graph reconstruction attack that can reconstruct a graph that has similar graph structural statistics to

the target graph. Shen et al. [7] further study the graph reconstruction attack. They improve the attack's ability to accurately recover edges only by touching the embedding matrix of the target graph. Chen et al. [1] propose an unlearning method to protect the graph data used in graph neural networks. Li et al. [4] survey the existing methods of watermarking the general deep neural networks. Zhang et al. [10] propose a method to use an entirely-randomized graph to watermark the graph neural networks.

## VI. CONCLUSION

In this paper, we propose an algorithm to watermark GNNs on cloud-based services. By employing the soft nearest neighbor loss, we retain the watermark on the model extracted by the model extraction attack. Furthermore, we propose an optimization-based method to improve the performance of the watermarked models. Extensive experiments on real-world datasets are conducted to illustrate the effectiveness of our method.

## ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback. This work is supported by the National Science Foundation of China (NSFC) under grant U1911401, NSFC under grant No.U1909207, and the Helmholtz Association within the project "Trustworthy Federated Data Analytics" (TFDA) (funding number ZT-I-001 4).

## REFERENCES

- [1] Min Chen, Zhikun Zhang, Tianhao Wang, Michael Backes, Mathias Humbert, and Yang Zhang. Graph unlearning. *arXiv preprint arXiv:2103.14991*, 2021.
- [2] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- [3] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International Conference on Machine Learning*, pages 3519–3529. PMLR, 2019.
- [4] Yue Li, Hongxia Wang, and Mauro Barni. A survey of deep neural network watermarking techniques. *Neurocomputing (Amsterdam)*, 461:171–193, 2021.
- [5] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. Tudataset: A collection of benchmark datasets for learning with graphs. In *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)*, 2020.
- [6] Yifei Shen, Yuanming Shi, Jun Zhang, and Khaled B Letaief. Graph neural networks for scalable radio resource management: Architecture design and theoretical analysis. *IEEE Journal on Selected Areas in Communications*, 39(1):101–115, 2020.
- [7] Yun Shen, Yufei Han, Zhikun Zhang, Min Chen, Ting Yu, Michael Backes, Yang Zhang, and Gianluca Stringhini. Finding mnemon: Reviving memories of node embeddings. *arXiv preprint arXiv:2204.06963*, 2022.
- [8] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction {API}s. In *25th USENIX security symposium (USENIX Security 16)*, pages 601–618, 2016.
- [9] Zhikun Zhang, Min Chen, Michael Backes, Yun Shen, and Yang Zhang. Inference attacks against graph neural networks. In *Proceedings of the 31th USENIX Security Symposium*, pages 1–18, 2022.
- [10] Xiangyu Zhao, Hanzhou Wu, and Xinpeng Zhang. Watermarking graph neural networks by random graphs. pages 1–6, Ithaca, 2021. IEEE.